

SRS DOCUMENT 4 — Notification & Alerts Module

(WhatsApp + Email + In-App Notifications)

Interns will **SUPPORT Shafvan**, and this will become a **core RhoFi communication subsystem**.

1. Executive Summary

The **Notification & Alerts Module** is responsible for delivering timely, actionable updates to RhoFi users across three channels:

- 1 In-App Notifications (Phase-1)
- 2 Email Alerts (Phase-1/Phase-2)
- 3 WhatsApp Alerts (Phase-2)

This system will provide a unified pipeline where internal modules (RSA, MF Engine, Watchlist, Blog Engine, Finance 360) can **push events**, and the notification engine will **format** → **queue** → **deliver** them.

Interns will support the build with:

- Templates
- Message formatting
- Queue logic
- In-app UI
- Structuring payloads

Primary backend implementation led by **Shafvan**, with design support from **Gopal**.

2. Business Problem

RhoFi needs a structured way to communicate:

- Trend alerts
- Research updates
- New blog drafts
- MF insights
- Spending alerts (Finance 360)
- System notifications (password resets, onboarding steps)

Currently, no centralized notification engine exists → leading to inconsistent communication, missed insights, and user disengagement.

A unified notification system creates:

- Better engagement
- Higher product stickiness
- Clear communication templates
- Multi-channel outreach capability

3. Module Objectives

- ✓ Deliver consistent alerts across channels
- ✓ Allow internal modules to push standardized notification events
- ✓ Provide template-driven messages

- ✓ Build a scalable queue-based delivery engine
 - ✓ Provide UI for users to view & manage notifications
 - ✓ Enable future expansion to push-based automation
-

4. User Personas

Persona	Role	Needs
User (Reader/Investor)	End User	Timely alerts, easy-to-read notifications
Founder (Sekar)	Strategy	Reliable communication pipeline
Backend Lead (Shafvan)	Architect	Stable, scalable notification system
Intern Developers	Support	Build templates, in-app UI, queue logic
Designer (Gopal)	UX	Clean notification UX

5. Functional Requirements

5.1 Notification Sources (Producers)

Notifications can originate from multiple RhoFi modules:

A. Automated Research & Blog Engine (ARBE)

- “New blog published”
- “Daily morning market note”

B. RSA Module

- Trendline break
- RSI oversold/overbought
- Pattern detection alerts

C. MF Module

- NAV updates
- Category insights
- SIP reminders

D. Finance 360

- Spending spike alert
- Low savings alert
- Net worth threshold alert

E. System-Level

- Password updated
 - Welcome onboarding steps
 - Subscription renewal upcoming
-

5.2 Notification Types

1. In-App Notifications

(Phase-1, Interns own UI)

- Title
- Short message
- Deep link (optional)
- Read/unread state
- Persisted in DB

2. Email Notifications

- Transactional emails

- Insights newsletter
- Blog publish alerts

3. WhatsApp Notifications

- Using template messages
- Opt-in required
- Custom payload structure

(Interns only prepare templates + payload schema; integration done by leads)

5.3 Notification Payload Structure

```
{
  "type": "insight",
  "channel": ["inapp", "email"],
  "title": "NIFTY up 1.2% today",
  "message": "NIFTY saw strong buying in IT and Banking...",
  "source": "MarketSummaryEngine",
  "deep_link": "/insights/market-summary",
  "user_id": "<UUID>",
  "created_at": "<timestamp>"
}
```

5.4 Notification Queue

The module must implement:

Queue States:

- Pending
- Processing
- Delivered
- Failed
- Retried (max 3 retries)

Intern task: Build in-app queue simulation and interface.

Lead handles actual infra queue (Redis, GCP Pub/Sub etc.).

5.5 Notification Preferences

Users must be able to toggle:

- Receive market alerts: On/Off
- Receive MF alerts: On/Off
- Receive newsletter: On/Off
- Receive WhatsApp: On/Off

Interns build settings UI.

5.6 Notification Center UI

Interns will build:

- Notification bell indicator
- Dropdown preview
- “View All” page
- Mark All Read
- Delete notification (optional)

Design support from Gopal.

6. Non-Functional Requirements (Enterprise Grade)

Category	Requirement
Scalability	Should handle 100K notifications/day
Performance	In-app notifications appear in <1 sec
Deliverability	Email delivery > 98%
Compliance	WhatsApp templates must follow policy; no advisory wording
Security	Prevent sending notifications to wrong users
Observability	Logs + delivery metrics

7. Data Model (DB Schema)

Table: notifications

Field	Type	Description
notification_id	UUID	PK
user_id	UUID	FK
title	string	Short title
message	text	Content
channel	string	inapp/email/whatsapp
type	string	system/insight/mf/market
source	string	module origin
deep_link	string	optional
status	string	delivered/unread/read
created_at	timestamp	created time

Table: notification_queue

Field	Type	Description
queue_id	UUID	PK
payload	JSON	full notification payload
status	string	pending/processing/failed
retries	int	retry count

Field	Type	Description
created_at	timestamp	queue time

Table: notification_preferences

| user_id | UUID

| receive_market_alerts | boolean

| receive_mf_alerts | boolean

| receive_blog_alerts | boolean

| receive_whatsapp | boolean

8. API Specifications

POST /notify/send

GET /notify/list?user_id=

POST /notify/mark-read

POST /notify/mark-all-read

GET /notify/settings

POST /notify/settings/update

9. User Flow Diagram

Signal → Notification Engine → Queue → Channel Processor → Delivery → UI
Display → User Reads

10. UI Wireframes (High-Level)

Notification Bell

- Shows unread count
- Opens dropdown

Notification List Page

- Filters (All, Insights, MF, System)
- Search bar

Settings Page

- Toggles for each alert type

Gopal will assist interns with final design polish.

11. Dependencies

Area	Contact
Backend	Shafvan

Area	Contact
UI/UX	Gopal
Infra	Boopalan
Business rules	Sekar & Rajani

12. Risks & Mitigation

Risk	Mitigation
Duplicate notifications	Add hashing to payload
High volume spikes	Use batching for email/WhatsApp
WhatsApp compliance	Keep templates pre-approved
User spam	Respect user preferences

13. Acceptance Criteria

- ✓ In-app notification UI fully working
 - ✓ Notification queue simulation works
 - ✓ Email templates ready
 - ✓ WhatsApp templates ready
 - ✓ Preference management working
 - ✓ Minimum 8 types of alerts defined
 - ✓ Published blog → triggers notification
 - ✓ Passed QA automation tests
-

14. 1-Month Delivery Plan

Week 1

- Understand notification architecture
- Create DB schema + UI wireframes
- Build in-app notification center skeleton

Week 2

- Implement notification queue logic
- Build templates for all events
- API integration

Week 3

- Notification settings
- Email payload logic
- WhatsApp template structure

Week 4

- Integrate with Blog Engine
- QA Automation
- End-to-end demo

KPIs

- 90% in-app notification functionality
 - 100% template coverage
 - End-to-end working demo with at least 1 real flow (Blog publish → alert)
-